

TABLE 5
COMMON ESP32-C3 RESET CAUSES

Reset Type	Description
POWER_ON	Device powered on normally
SOFTWARE_RESET	Reset initiated by the firmware or software
WATCHDOG	Reset triggered by the watchdog timer due to system unresponsiveness
DEEP_SLEEP_WAKE	The device woke up from deep sleep mode
BROWNOUT	Reset caused by a drop in the supply voltage below the safe operating level
PANIC	Reset caused by a fatal runtime exception or system crash
Other	Any other reset reason not covered by the above categories

```
float getFlashUsedPercent() {
  uint32_t flashSize = ESP.getFlashChipSize();
  uint32_t sketchSize = ESP.getSketchSize();

  if (flashSize == 0) return 0;

  return ((float)sketchSize / flashSize) * 100.0;
}
```

Fig. 7: Function used to calculate the percentage of flash memory

```
const char* getResetReasonText() {
  switch (esp_reset_reason()) {
    case ESP_RST_POWERON: return "POWER_ON";
    case ESP_RST_SW: return "SOFTWARE_RESET";
    case ESP_RST_WDT: return "WATCHDOG";
    case ESP_RST_DEEPSLEEP: return "DEEP_SLEEP_WAKE";
    case ESP_RST_BROWNOUT: return "BROWNOUT";
    case ESP_RST_PANIC: return "PANIC";
    default: return "OTHER";
  }
}
```

Fig. 8: Function to identify and return the ESP32-C3 reset reason

```
String wifiChText = "CH" + String(WiFi.channel());
```

Fig. 9: Function to retrieve the Wi-Fi channel

CPU usage from the FreeRTOS idle task runtime statistics.

RAM usage (%)

RAM usage indicates the percentage of the ESP32-C3 Super Mini's dynamic memory (heap) currently occupied by the firmware during operation. The heap is used to allocate memory for variables, buffers, network communication, and other runtime processes. RAM utilisation is calculated using the following equation:

```
RAM usage (%) = ((total heap - free heap) / total heap) * 100
```

where total heap is the total heap memory available to the application, and free heap is the heap memory currently available for allocation.

The getRamUsagePercent() function reads the total and available heap memory, calculates the percentage currently in use, constrains the result to the valid range of 0-100%, and returns the RAM usage percentage. Fig. 4 shows the getRamUsagePercent() function used to calculate ESP32-C3 heap memory utilisation.

RSSI dBm

RSSI (received signal strength indicator) represents the strength of the Wi-Fi signal received by the ESP32-C3. It is measured in dBm (decibels

referenced to one milliwatt), where values closer to 0dBm indicate a stronger signal and more negative values indicate a weaker signal. Table 3 shows the typical Wi-Fi signal strength ranges.

Monitoring RSSI remotely helps determine whether communication problems are caused by poor Wi-Fi signal strength. The WiFi.RSSI() function retrieves the current RSSI value of the connected wireless network, enabling the ESP32-C3 to continuously monitor the quality of its Wi-Fi connection. Fig. 5 shows the code used to read the Wi-Fi

```
uint32_t uptime = millis() / 1000;
String uptimeDHMS = getUptimeDHMS(uptime);
.....

String getUptimeDHMS(uint32_t seconds) {
  uint32_t days = seconds / 86400;
  seconds %= 86400;

  uint8_t hours = seconds / 3600;
  seconds %= 3600;

  uint8_t minutes = seconds / 60;
  uint8_t secs = seconds % 60;

  char buffer[20];
  sprintf(buffer, "%03lu:%02u:%02u:%02u",
    days, hours, minutes, secs);

  return String(buffer);
}
```

Fig. 10: Function to calculate and format the ESP32-C3 uptime

```
void publishLedStatus() {
  if (ledState) {
    client.publish(led_status_topic, "ON", true);
  } else {
    client.publish(led_status_topic, "OFF", true);
  }
}
```

Fig. 11: Function to publish the LED on/off status to the MQTT broker

```
if (String(topic) == led_control_topic) {

  if (message == "ON") {
    digitalWrite(LED_PIN, LOW);
    ledState = true;
    publishLedStatus();
  } else if (message == "OFF") {
    digitalWrite(LED_PIN, HIGH);
    ledState = false;
    publishLedStatus();
  }
}
```

Fig. 12: Callback function to receive MQTT commands and control the onboard LED

```
if (String(topic) == esp32c3_reset_topic) {
  if (message == "RESET") {
    delay(1000);
    ESP.restart();
  }
}
```

Fig. 13: Callback function to receive the MQTT reset command

signal strength (RSSI) of the connected network.

Chip temperature (°C)

The ESP32-C3 includes an internal temperature sensor that provides an